Advanced Algorithms (41052)

Assignment 2

Ben Mackaway (25317696)

Bradley Wilson (25929112)

CJ Kornie (2482287)

1. The problem and the algorithms **3**
Fermat 3
Miller-Rabin 4
AKS 4
2. Theoretical comparison and predictions **5**
Fermat 5
Miller-Rabin 5
AKS 6
Prediction 6
3. Experimental methodology **7**
Input 7
Measurement 8
Benchmark 8
Fairness 9
4. Results and analysis **10**
Correctness 10
Runtime 10
Extra Rounds 12
Error Rate 13
Variation 14
AKS is slow 16
AKS against randomised Tests 17
Did we predict well? 18
5. What surprised you **19**
6. What we'd do next **20**
7. Individual contributions **21**
Ben 21
Brad 21
CJ 21
8. AI use **22**
References 23

1. The problem and the algorithms

The problem we are trying to solve is Primality testing. In this case, measuring the runtime and accuracy differences of deterministic and random algorithms. The problem is defined as such:

Given an integer n ≥ 2, decide whether n is prime, meaning that its only positive divisors are 1 and n.

In our implementations n is an unsigned 64-bit integer, so 2 ≤ n < 2^64. The size of the input is the number of bits in n, about log_2 n, not n itself. An efficient algorithm must run in polynomial time in log n. Trial division, which tries every divisor up to √n, takes about √n = 2^(b/2) steps for a b-bit number. That is exponential in the input size, and it is why trial division is not one of the algorithms compared.

The three algorithms below fall into two kinds:

- Randomised tests (Fermat and Miller-Rabin). These are Monte Carlo algorithms with one-sided errors. If n is prime, the result is always "prime". If n is composite, the result is "prime" with some small probability, which shrinks as more rounds k are run.
- A deterministic test (AKS). It uses no randomness and is always correct but generally much slower to run, even when compared against multiple round randomized algorithms.

Fermat

The Fermat test is based on Fermat's little theorem. This states if n is prime, then a^(n-1) ≡ 1 (mod n) for every a not divisible by n. Each round has a random base from 2 to n - 2, and computes a^(n-1) mod n. If the result is not 1, n is certainly composite and the tests stop. If it is 1, the test returns prime.

Every round uses one modular exponentiation, so a full test is O(k log n) modular multiplications, or O(k log^3 n) bit operations. Composites generally return in the first round, so the test normally stops after a single exponentiation, while primes always need all k rounds.

The weakness of Fermat is a type of composites called Carmichael numbers (for example 561 = 3 * 11 * 17). For these, a^(n-1) ≡ 1 (mod n) returns prime for every base a coprime to n, so a test returns “prime” unless the chosen base happens to share a factor with n. For a Carmichael number where prime factors are large, almost every base is coprime to n, so the chance of a wrong return value remains high no matter how many iterations are run. Every other composite fails a round for at least half of the bases coprime to n, so its error falls to at most 2^-k after k rounds. Alford, Granville and Pomerance (1994) proved that there are infinitely many Carmichael numbers, so the Fermat test has no error bound that holds for every input.

Miller-Rabin

Miller-Rabin strengthens the Fermat test by checking how a^(n-1) reaches 1, not only that it does. Write n - 1 = 2^s · d with d odd, and look at the chain a^d, a^(2d), a^(4d), ..., a^(n-1) (mod n). If n is prime, 1 and -1 are the only square roots of 1 mod n, so the chain must start at 1 or hit n - 1 before the end. A chain that reaches 1 from any other value has found a non-trivial square root of 1, so n is composite. A base a that exposes n this way is called a witness.

One round picks a random base a in [2, n - 2], computes a^d, then squares up to s - 1 times. For example, take n = 561 = 3 · 11 · 17 and base 2. Here 560 = 2^4 · 35 and the chain is 263, 166, 67, 1. It reaches 1 from 67 rather than from 560, so 561 is composite. Fermat only sees 2^560 ≡ 1 (mod 561) and passes it.

The guarantee is one-sided. “Composite” is always correct, and a prime always passes. “Prime” is wrong with probability at most 4^-k on any odd composite, because at least 3/4 of bases are witnesses. Unlike Fermat, no class of composites defeats it, Carmichael numbers included. At k = 9 the bound is 4^-9 ≈ 3.8 x 10^-6.

A round is one modular exponentiation, about b squarings and b/2 multiplies for a b-bit n, and the squaring chain reuses those squarings. So k rounds cost O(k · b) fixed-width multiplies, or O(k log^3 n) bit operations, with O(1) memory, the same as Fermat. Composites usually exit in round 1 and primes always run all k rounds.

AKS

The AKS test (Agrawal, Kayal and Saxena, 2004) was the first primality test that is general, deterministic and provably polynomial-time. It rests on one identity: for an integer n ≥ 2 and a coprime to n, n is prime exactly when (x + a)^n ≡ x^n + a (mod n). Checking this directly would mean expanding a polynomial with n coefficients, which is far too slow. AKS instead works in the ring (Z/nZ)[x]/(x^r − 1), where every polynomial has only r coefficients, and checks the identity for a small set of values of a.

The algorithm has six steps. (1) If n = a^b for some b ≥ 2, return composite. (2) Find the smallest r such that the multiplicative order of n modulo r exceeds $(\log_2 n)^2$. (3) If any a ≤ r has 1 < gcd(a, n) < n, return composite. (4) If n ≤ r, return prime. (5) For each a from 1 to $\lfloor \sqrt{\varphi(r)} \cdot \log_2 n \rfloor$, check (x + a)^n ≡ x^n + a in the ring above, and return composite on the first failure. (6) Otherwise return prime.

Each step exists because the correctness proof requires it. The perfect-power check removes prime powers, which the main theorem does not cover. The gcd check and the n ≤ r case guarantee that gcd(n, r) = 1 and that n is larger than r. The bound on r limits how many values of a must be tried. Reducing modulo x^r − 1 keeps each polynomial at r coefficients instead of n. Unlike Fermat and Miller-Rabin, AKS makes no random choices and has no error probability. It is also the only one of the three algorithms that needs more than constant memory.

2. Theoretical comparison and predictions

Fermat

Fermat has a time complexity of O(k log^3 n) bit operations, since each of its k rounds is one modular exponentiation of O(log n) multiplications on numbers of about log n bits. In our implementation every number is a 64-bit word, so each multiplication takes a fixed number of CPU instructions regardless of the size of n. We therefore expect the running time to behave more like O(k log n), growing roughly in proportion to the number of bits rather than with its cube.

Fermat should have the smallest time complexity per iteration as each is a single modular exponentiation. It also only needs a few 64-bit variables so its memory use is constant.

Its running time should mostly depend on whether n is composite. If it is, it would usually return instantly, but if not, it will run through every round up until the k value. For most composites the first base already gives a result other than 1, so the test stops after one round and should take about 1/k of the time of a prime of the same size. Raising k should therefore slow down primes in proportion to k while leaving most composites unchanged.

Miller-Rabin

Miller-Rabin does the same modular exponentiation per round as Fermat, so it has the same time complexity of O(k log^3 n) bit operations. The only extra work is the squaring chain, which reuses the squarings the exponentiation already does. With 64-bit words each multiplication takes a fixed number of instructions, so we expect running time to grow roughly in proportion to the number of bits (a log-log slope between 0.8 and 1.3) and linearly in k.

We therefore expect one round to cost 1.0 to 1.1 times a Fermat round, with constant memory like the others. Its running time should mostly depend on whether n is prime. Primes always run all k rounds, so a prime at k = 9 should be about 9 times slower than one Fermat round, while a composite usually finds a witness in round 1 and should be 6 to 9.5 times cheaper than a prime.

AKS

The original analysis of AKS gives O(log^12 n) bit operations. Our implementation uses schoolbook polynomial multiplication at O(r^2) per squaring, with r ≈ (log n)^2, about (log n)^2 values of a and about log n squarings per value. This gives about log^7 n coefficient operations, each constant cost on 64-bit words. We therefore predict a log-log slope near 7 against bit length, far steeper than the near-linear growth expected for Fermat and Miller-Rabin. Memory is O(r) = O(log^2 n) words, a few kilobytes at the sizes we can run. It is small but not constant like the other two.

AKS also has three early exits: the perfect-power check, the gcd loop for composites with a small factor, and the n ≤ r case for tiny n. Only primes (and composites with no factor below r) reach step 5 and pay for every value of a. We therefore expect a prime to cost hundreds of times more than a composite of the same size. This is the opposite pattern from Fermat and Miller-Rabin, where composites are cheap and only primes pay for all k rounds.

Prediction

- **1:** Fermat and Miller-Rabin cost about the same per round (Miller-Rabin 0-10% slower). *Blind.*
- **2:** On 64-bit inputs, a composite is 6-9.5x cheaper than a prime at k = 9. *Blind.*
- **3:** Fermat passes essentially all Carmichael numbers whatever k is; Miller-Rabin does not. *Blind.*
- **4:** Miller-Rabin error never exceeds 4^{-k}: (a) no errors on random composites in 10^6 calls at k = 1; (b) a mean single-round error of 0.01-0.15 on Carmichael numbers; (c) no composite n ≤ 20,000 above 0.25. *Blind.*
- **5:** AKS time on primes grows with a log-log slope of about 7. *Informed.*
- **6:** AKS is at least 10^4x slower than Miller-Rabin at 64 bits, with no crossover. *Blind.*
- **7:** Fermat and Miller-Rabin grow roughly linearly in bit length (slope 0.8-1.3). *Blind.*
- **8:** On primes, time is linear in k; k = 9 costs about 9x one round. *Blind.*
- **9:** Fermat is marginally faster than Miller-Rabin in practice. *Blind.*
- **10:** An AKS prime costs hundreds of times a same-size composite. *Informed.*
- **11:** AKS uses O(log^2 n) memory; the other two use constant memory. *Blind.*
- **12:** AKS is never wrong, Fermat is wrong on edge cases such as Carmichael numbers, and Miller-Rabin is wrong far less often. *Blind.*

3. Experimental methodology

Input

Using a script (scripts/gen_data.py), we generate 50,886 input numbers from a fixed seed and these numbers are shared across all algorithms. These are stored and inputted as 64-bit unsigned integers. These are labelled prime or composite by a separate python script which uses deterministic Miller-Rabin using the first 12 primes as bases.

Type	Count	Why
Random Primes	15,000	Needed to test on random primes
Random (odd) Composites	15,000	Needed to test on random non-primes
Semiprimes pq, with p and q of similar size	15,000	No small factor, so AKS cannot finish early at its gcd step
Prime squares	1,500	Rejected at AKS's first step (perfect-power check
Carmichael numbers	1,777	Composites that pass a Fermat round for every base coprime to n. All 105 below 10^7, plus larger ones of the form (6m+1)(12m+1)(18m+1)
Strong pseudoprimes	169	Composites that pass Miller-Rabin rounds with particular fixed bases
p(2p - 1) with p prime, p ≡ 3 (mod 4)	2,440	The composites closest to Miller-Rabin's worst case

These types were chosen, rather than wholly random number generation, to better demonstrate the comparative strengths and weaknesses of the different algorithms, we include more edge cases in the total input dataset. Primality dictates whether Fermat and Miller-Rabin can stop early after the first round. Carmichael numbers are the worst case for Fermat. The pseudoprimes and the p(2p - 1) family are the hardest inputs for Miller-Rabin, whose error per round is proven to be at most 1/4. We found the p(2p - 1) form by checking every base for every odd composite up to 20,000. The worst case was n = 12,403 = 79 * 157, where 24.5% of bases returned prime.

Measurement

We created a benchmark script which runs multiple experiments in sequence. We measure:

- Execution time for each bit length
- Execution time for the number of rounds (1-20) for randomized algorithms using 64-bit input numbers.
- Running duplicate randomized tests with different drawn bases to compare the average call with the worst 1% of cases.
- How often each randomised test returns prime for a composite.
- How often a single round of Fermat or Miller-Rabin returns prime for each Carmichael number, estimated from 2,000 trials per number.
- For all composites under 20,000, we try every base and count the percentage of them which wrongly return a prime.
- How long each step of AKS takes.

Unless k is being varied, the randomised tests use k = 9 rounds. This bounds Miller-Rabin's error at 4^-9 ≈ 3.8 * 10^-6. The error-rate experiment runs each call for up to 20 rounds and records how many rounds returned prime, so a single run gives the error rate at every k from 1 to 20.

AKS cannot be run at 64 bits in a reasonable time, so its sweeps are smaller. We time primes of 8-26 bits, random composites up to 40 bits and balanced semiprimes of 30–35 bits, with three instances per size and a 40-second cap per call. Fermat and Miller-Rabin are also run on these same instance files, so the head-to-head comparison uses identical inputs. For AKS we additionally record per-step times (steps 1, 2, 3 and 5), the chosen r, the number of values of a checked, and the number of polynomial squarings. These let us explain AKS’s cost rather than only measure it.

Benchmark

Modern processors change their clock speed based on temperature and load, this would change the speed at which an algorithm runs midway through a benchmark. To remedy this, during benchmarking we disable boosting and fix the processor to its base clock (In this case, 4.7 GHz). The benchmark is also pinned to a single CPU core (core 2, using taskset), and core 0 is avoided because it usually handles most hardware interrupts.

Times were taken with C++'s std::chrono::steady_clock, which measures in steps of 10 ns. A single Fermat or Miller-Rabin call on a composite takes only 20 to 350 ns, close to the step size. So instead of timing calls one at a time, we timed a batch of all 1,000 inputs of one family and size and divided it by 1,000. Before timing, every batch was run once without timing to ensure the state of the CPU cache did not affect results. Each batch is then run 21 times and the median is reported. We also interleave the different methods along the benchmark runtime (by batch).

Over 3 identical runs, the benchmark reported a median 0.5% difference in timing and at most 10% in outliers. Seeds are fixed so this variance is due to processor timing differences (such as branch prediction, dual threading, or I/O interrupts) but over many runs averages out. For primes and typical composites the spread did not change, because the work per call

does not depend on which base is chosen. The spread grows only for the p(2p - 1) input type, where the base determines which round n is found composite. Therefore, we will treat differences of a few percentage points as noise and will only report significant differences in percentages over multiple runs.

AKS calls run for milliseconds to seconds, so the batching used for Fermat and Miller-Rabin is unnecessary. Each AKS call is timed on its own with std::chrono::steady_clock, whose 10 ns resolution is negligible at that scale.

Fairness

Every algorithm uses the same input dataset and the same random seeds. We also define a shared implementation of modular multiplication and exponentiation using a 128 bit intermediary. For each type and size, the three algorithms are timed one after another, so any slow change in the machine's state affects all of them equally. Only the primality calls are timed, the script to check results, read data, and write the results do not affect measurements.

Before benchmarking, a test program checked every algorithm against independent results in 503,411 checks, covering:

- every n up to 200,000 (3,000 for AKS)
- lists of known primes, Carmichael numbers and pseudoprimes
- the labelled data set.

All three algorithms' results were checked against an independent sieve of primes (for n up to 200,000, or 3,000 for AKS) and the labelled data set. Every final measurement was taken on a single machine, an AMD Ryzen 7 9800x3D running Linux, compiled with GCC 16.2.1 at -O3 (CMake Release), with the same instance files for all three algorithms.

4. Results and analysis

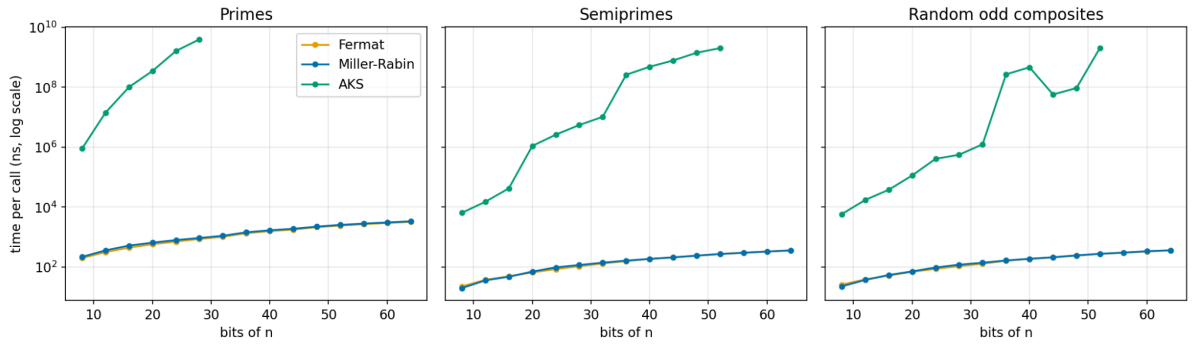
Correctness

Each label comes from the deterministic Miller-Rabin check in a separate python script, which was itself checked against a sieve up to 2 x 10^6. Every integer up to 200,000 for the randomised algorithms, and up to 3,000 for AKS, was also checked against a sieve.

Fermat returned prime for 1,696 composites during testing, this was expected and the majority of which were the set Carmicheal numbers and other pseudoprimes. There was one case where during the randomized run a single error occurred, but it was later determined this was a randomly generated Carmichael number (2821).

The other algorithms never returned incorrect results, and all algorithms never returned composite for a prime.

Runtime



Algorithm	Input	16 bits	32 bits	64 bits
Fermat	prime	440 ns	1.01 µs	3.19 µs
Miller-Rabin	prime	515 ns	1.10 µs	3.34 µs
Fermat	semiprime	48.5 ns	131 ns	360 ns
Miller-Rabin	semiprime	47.0 ns	138 ns	354 ns
AKS	prime	101 ms	-	-
AKS	semiprime	42.1 µs	10.3 ms	-

Fermat and Miller-Rabin. On primes, the running time of both tests grows with a log-log slope of about 1.4 (1.44 for Fermat, 1.37 for Miller-Rabin). A slope of 1 means the time doubles when the bit length doubles. The usual O(k log^3 n) bound predicts a slope of 3, but that bound assumes that multiplying two log n-bit numbers costs log^2 n.

In our implementation every number fits in a single 64-bit word. Each modular multiplication is one 128-bit product followed by one remainder, so it takes roughly the same time at any size. The only cost that grows with n is the number of multiplications in the exponentiation, which is proportional to the bit length. That alone gives a slope of 1. The rest of the growth most likely comes from the hardware division behind the remainder, which takes longer for larger operands.

Fermat compared with Miller-Rabin. The two tests do almost the same operations. Miller-Rabin computes a^d and then squares it s times, which is the same number of squarings as Fermat's a^(n-1). The extra work in Miller-Rabin is splitting n - 1 into 2^s*d and comparing the value against 1 and n - 1 after each squaring.

On primes, where all nine rounds run, this makes Miller-Rabin about 5% slower at 64 bits and 10-17% slower at 8-16 bits. The gap is larger at small sizes because the exponentiation is shorter there, so the fixed extra work makes up a bigger share of each round. On composites both tests usually run only one round, and the times differ by about 1%. That is too small to separate from the variation between runs described in section 3.

Primes compared with composites. A 64-bit prime takes 3.19 µs and a 64-bit composite takes about 356 ns, a ratio of almost exactly 9, which equals k. This is because a composite almost always returns composite in the first round, while a prime must pass all k rounds.

AKS shows the same pattern in a more extreme form:

- Prime squares take a constant 380 ns at every size, because the perfect-power check in step 1 identifies them.
- Most random composites have a small factor, which is found by the gcd in step 3.
- Semiprimes reach step 5 from 20 bits up, but the first polynomial check fails.

AKS. On primes, AKS grows with a slope of 6.78 between 12 and 26 bits. The structure of the algorithm predicts about 7, built up as follows:

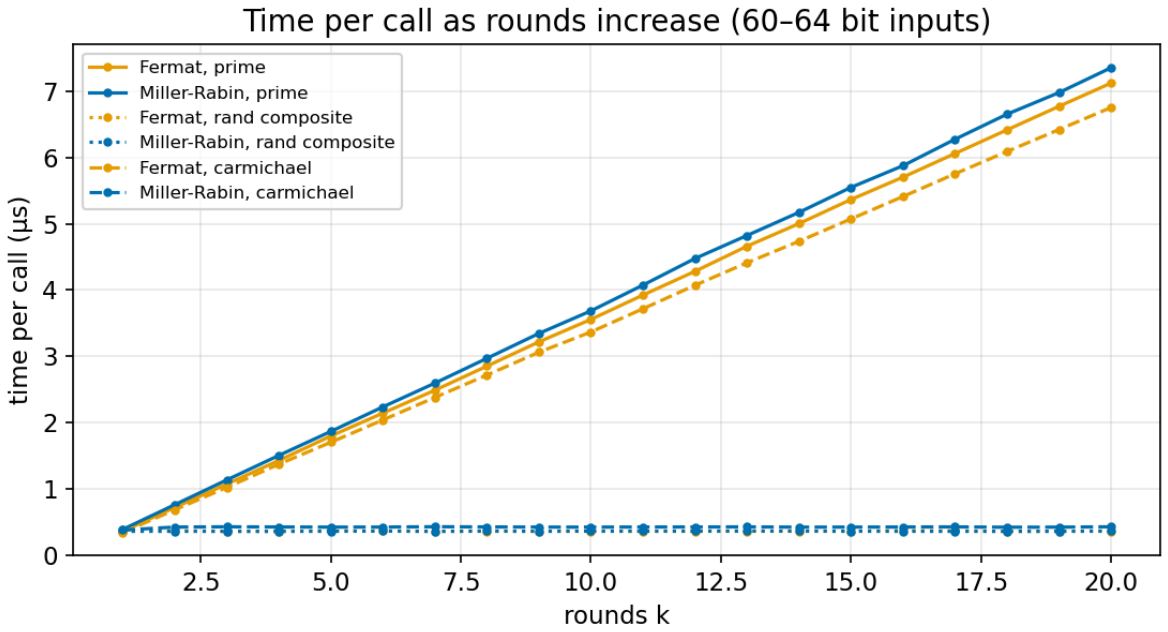
- The measured r is close to log^2 n.
- The number of values of a to check is $\sqrt{\varphi(r)}*\log n$, which is also about log^2 n.
- Each check takes about log n squarings of a polynomial with r coefficients.
- Each squaring takes r^2 coefficient multiplications.
- Multiplying these gives log^2 n * log n · log^4 n = log^7 n.

At 28 bits AKS is about 4.3 million times slower than Miller-Rabin, and a 24-bit prime already takes 1.29 s. The two slopes differ by about 5.4, so each doubling of the bit length makes the gap about 2^5.4 ≈ 42 times larger. We cut off runtime for a single run to 2s, as such, AKS runs for greater numbers of bits are not represented.

The AKS semiprime line in the figure above also jumps sharply between 32 and 33 bits. This comes from our implementation rather than from the algorithm, and it is discussed in section 5.

Extra Rounds

Extra Rounds: time per call against k on 64-bit inputs. Primes and Carmichael numbers under Fermat rise linearly; composites are flat because they exit in round 1.



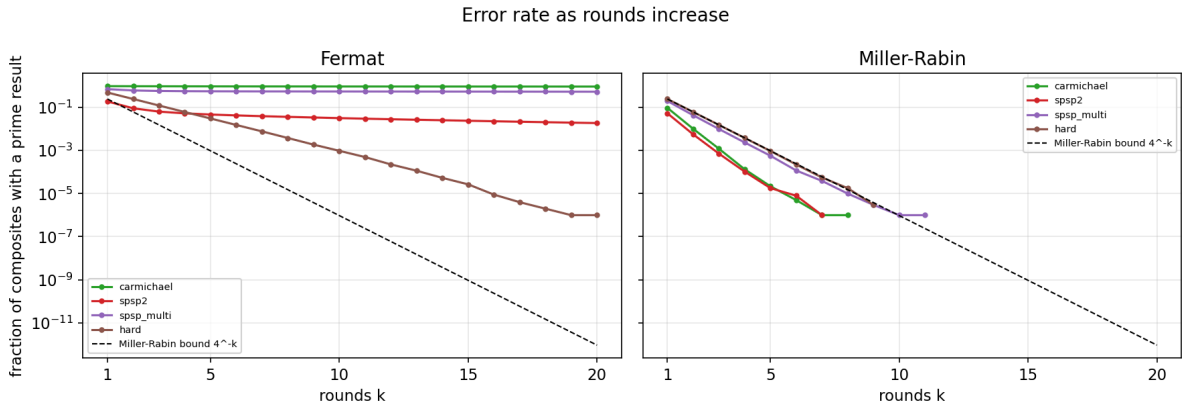
Algorithm	Input	k = 1	k = 9	k = 20	k = 20 = k = 1
Fermat	Prime	358 ns	3.22 µs	7.12 µs	19.9
Miller-Rabin	Prime	385 ns	3.34 µs	7.36 µs	19.1
Fermat	Random Composite	357 ns	357 ns	356 ns	1.0
Miller-Rabin	Random Composite	357 ns	357 ns	361 ns	1.0
Fermat	Carmichael	339 ns	3.06 µs	6.75 µs	19.9
Miller-Rabin	Carmichael	374 ns	420 ns	424 ns	1.1

On primes, the time grows linearly in k for both tests, because every round has to run. Miller-Rabin can be seen in both figures to be slightly slower due to the higher overhead (1.1x) but the difference is negligible. On random composite numbers, the difference is almost nonexistent and flat regardless of k value. This is because the call generally returns after the first iteration. For typical inputs (composites), the extra rounds do not affect runtime.

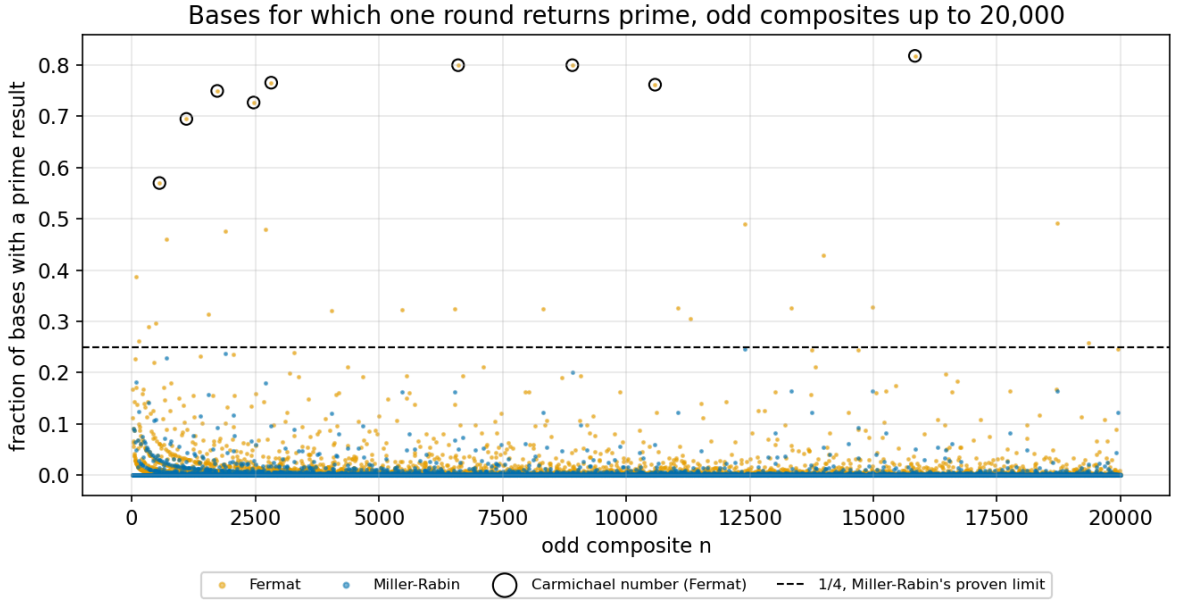
Carmichael numbers demonstrate the inaccuracy of Fermat in comparison to Miller-Rabin, as these are treated the same as primes requiring multiple iterations in order to achieve an incorrect result.

Error Rate

Error Rate: fraction of calls returning “prime” for composites, by k. Fermat’s Carmichael curve stays near 1, and Miller-Rabin’s tracks 4^−k.

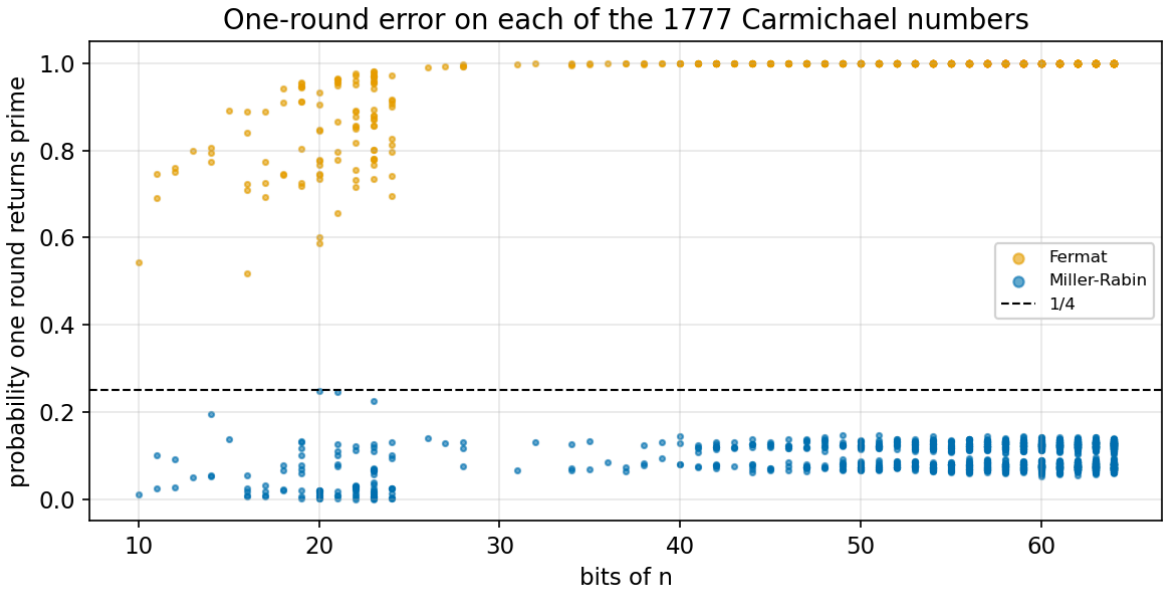


On random 64-bit composites and semiprimes, neither test returned a wrong result in a million calls, even with one round. For a semiprime n = pq, the number of bases for which a Fermat round returns prime is gcd(p - 1, n - 1) * gcd(q - 1, n - 1). For random p and q this is usually tiny compared with n. The 4^-k bound is a worst case over all composites, so it greatly overstates the error on typical inputs. Composites that pass a Miller-Rabin round with base 2 also gave an error of only 5.4% with one round.



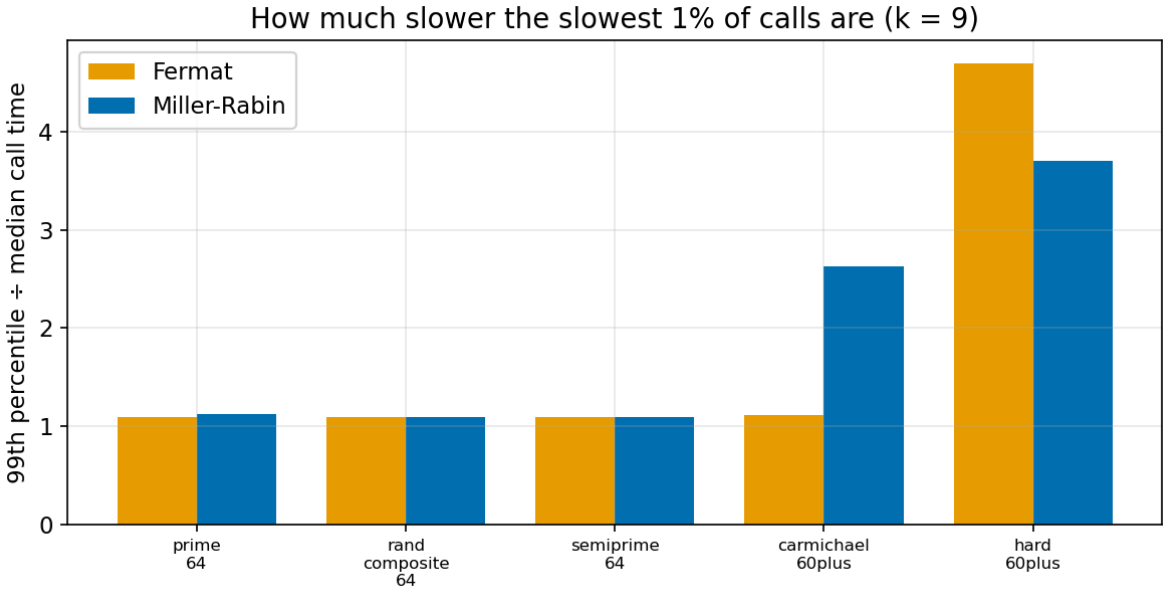
The figure above shows the one-round error percentage for each Carmichael number below 20k. These are clearly far above the rest of the points and are not present in the Miller-Rabin

numbers. This can be seen more clearly in the figure below where every single Carmichael number resides above the 1/4th Miller-Rabin proven limit.



Variation

Variation: p99 divided by median over repeated calls. The p(2p − 1) family is the widest because the chosen base decides which round finds the witness.



Algorithm	Input	Mean k	Median	99th %	p99 + median	Wrong
Fermat	prime	9.000	3.24 µs	3.54 µs	1.09	0

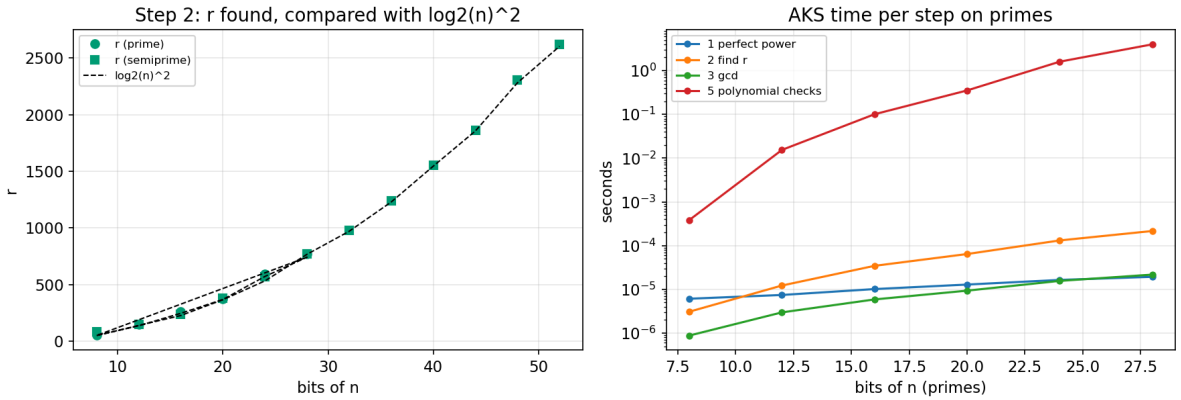
Miller-Rabin	random composite	1.000	410 ns	450 ns	1.10	0
Fermat	Carmichael	9.000	3.10 µs	3.44 µs	1.11	199,999
Miller-Rabin	Carmichael	1.113	400 ns	1.05 µs	2.62	0
Fermat	p(2p - 1)	1.992	550 ns	2.58 µs	4.69	388
Miller-Rabin	p(2p - 1)	1.332	400 ns	1.48 µs	3.70	1

For the typical random numbers (both primes and composites), every call has the same number of iterations, this being nine for a prime, and one for a composite. The 99th percentile is only about 10% above the median and this is mostly due to the CPU and 10ns clock resolution rather than the algorithm.

The spread only appears on inputs where many bases give a round that returns prime. On p(2p - 1), a Fermat round returns prime with probability 1/2, so the number of rounds follows a geometric distribution with mean 1/(1 - 1/2) = 2. The measured mean was 1.992. For Miller-Rabin the probability is 1/4, giving a mean of 4/3, and the measured mean was 1.332. The 1% of calls that draw several such bases in a row take 3.7-4.7 times the median.

The cost of randomness in time is bounded, because no call can run more than k rounds. Even the slowest calls on p(2p - 1) take about as long as a prime. Fermat on Carmichael numbers has no spread at all, since all nine rounds run every time and the result is wrong in 199,999 of 200,000 calls. AKS makes no random choices, so it does the same work every time it is given the same input.

AKS is slow



Outside of certain easy cases handled with integer operations such as perfect powers, almost all of the running time of AKS goes to the polynomial check (step 5). The figure above shows a single call at each bit length, with the time divided between the steps.

On primes, step 5 takes 97.4% of the time at 8 bits and 100.0% from 16 bits onwards. Steps 1 to 4 are a perfect-power check, a search for r, a gcd for each value up to r, and a single comparison, so their cost grows slowly with n. In step 5 every value of a up to $\sqrt{\varphi(r)}*\log n$ is checked. Each check raises a polynomial with r coefficients to the power n, which takes about log n squarings of r^2 coefficient multiplications each. For a prime, every value of a passes, so all of these checks run. At 28 bits this means 755 polynomial exponentiations, with r = 769.

Input	Bits	r	log^2 n	Value	Step 5 %	Total
prime	16	263	247	255 of 255	100.0%	101 ms
prime	28	769	742	755 of 755	100.0%	3.99 s
semiprime	16	239	227	None (step 3)	0%	46.1 µs
semiprime	28	773	768	1 of 770	94.9%	5.6 ms
semiprime	52	2621	2604	1 of 2612	99.9%	2.04 s

The table above also shows that r is always just above log^2 n. The search in step 2 starts at small values and stops at the first r for which the order of n modulo r exceeds log^2 n. In practice such an r turns up soon after log^2 n. The proof of AKS only guarantees that r = O(log^5 n). Because the actual r is much smaller, the measured growth is close to log^7 n rather than the log^10.5 n or log^12 n given by the published bounds.

On composites, the cost depends on the step at which the result is determined. Semiprimes up to 16 bits have a factor below r, which is found by the gcd in step 3, so step 5 never runs. From 20 bits onwards both factors are larger than r and step 5 is reached, but the congruence fails for the first value of a. Only one of the 770 required checks runs at 28 bits.

As a result, a 28-bit semiprime (5.6 ms) is about 700 times faster than a 28-bit prime (3.99 s), close to the 755 values needed for a prime.

For the randomised tests, the ratio between primes and composites is fixed at k. For AKS it grows with n, because the number of values needed for a prime grows as log^2 n, while a composite almost always fails at the first one.

On primes of 12-26 bits, time grows as bits^6.8 (log-log least squares, 8 sizes, 3 primes each), close to the predicted 7. The 52-bit semiprime gives a concrete anchor for how far this reaches. One check takes 2.04 s, and a 52-bit prime would need all 2,612 of them, so roughly 1.5 hours for a single prime. Larger primes are not practical to run, and any AKS figure at 64 bits or above elsewhere in this report is an extrapolation.

AKS against randomised Tests

Bits	AKS	Fermat (k=9)	Miller-Rabin (k=9)	AKS ÷ Miller-Rabin
8	892 µs	197 ns	216 ns	≈ 4×10^3
16	101 ms	440 ns	515 ns	≈ 2×10^5
24	1.63 s	701 ns	788 ns	≈ 2×10^6
28	3.99 s	859 ns	926 ns	≈ 4.3×10^6

There is no crossover between AKS and the randomised tests, and the gap grows with input size. On 16-bit primes AKS takes 101 ms against 515 ns for Miller-Rabin, a factor of about 2 * 10^5. At 28 bits the factor is 4.3 * 10^6. The gap widens because the algorithms scale differently. AKS grows with a log-log slope of about 6.7, while Fermat and Miller-Rabin grow with a slope of about 1.4, so each doubling of the bit length widens the gap by roughly 2^5.3 ≈ 40 times. AKS is not slow per operation. At about 0.3 ns per coefficient multiply-add it is as cheap as any inner loop can be. It is slow because it needs about log^7 n of those operations, while a randomised test needs only k modular exponentiations. Composites narrow the gap but never close it. A 32-bit balanced semiprime costs AKS 10.3 ms, about 7.5 * 10^4 times the 131-138 ns of the randomised tests, because the first polynomial check fails and AKS exits early, but the randomised tests exit early too. We cannot run AKS at 64 bits, so any 64-bit comparison is an extrapolation. Extending the b^6.78 fit gives about 1,000 s, but that fit lies entirely below 2^32 and underestimates a 52-bit prime by about twenty times (about 1.5 hours measured against 230 s fitted). The 64-bit gap of at least 3 x 10^8 is therefore a lower bound. Prediction P6 (AKS at least 10^4 times slower, no crossover) held by a wide margin, and the 10^4 threshold was about 400 times too low at 28 bits.

Did we predict well?

Eight hits, two partial, one near-miss, one untested. Each is judged against the wording above.

- **1, Partial:** Miller-Rabin was +4.7% at 64 bits and ≤8% from 28 bits, but +13-17% at 12-24 bits.
    - Its fixed extra work is a bigger share of a short exponentiation.
- **2, Hit:** 8.9x for Fermat and 9.4x for Miller-Rabin (7.9x when each call is timed alone).
    - The ratio is about k because composites exit in round 1.
- **3, Hit:** Fermat was wrong on 0.990 of calls at k = 1 and 0.960 at k = 9 over all Carmichael numbers, and on 199,999 of 200,000 above 60 bits. Miller-Rabin was 0.097 at k = 1 and 0 of 10^6 at k = 9.
    - The fall to 0.960 comes from small Carmichael numbers like 561, where some bases share a factor with n.
- **4, Hit:** (a) 0 errors in 10^6 calls; (b) mean 0.0971; (c) maximum 0.2452 at 12403 = 79 · 157.
    - This is a theorem (Rabin, 1980), so a miss would have meant a bug. The p(2p − 1) family reaches 0.2499 at k = 1 and 3 in 10^6 at k = 9, against a bound of 3.81 x 10^{-6}.
- **5, Hit:** slope 6.78 over 12-26 bits (6.68 over 12-28).
    - The measured r stays just above log^2 n, as the prediction assumed.
- **6, Hit on direction:** the gap is 4 x 10^3 at 8 bits and 4.3 x 10^6 at 28 bits, with no crossover.
    - AKS cannot be run at 64 bits, so that figure is an extrapolation, and the 10^4 threshold was about 400x too low at 28 bits.
- **7, Near-miss:** slopes of 1.44 (Fermat) and 1.37 (Miller-Rabin) over 12-64 bits.
    - Inside the range below 2³^2 (1.20 and 1.12) and outside above it (1.54 and 1.50), where products stop fitting in 64 bits. The prediction did not name a fit range.
- **8, Hit:** k = 9 over k = 1 was 9.0 (Fermat) and 8.7 (Miller-Rabin); k = 20 over k = 1 was 19.9 and 19.1.
    - Every round does the same work on a prime.
- **9, Partial:** Fermat was faster on primes (4.6% at 64 bits), equal on random composites, and about 7x slower on Carmichael numbers.
    - It runs every round before returning a wrong answer, so “which is faster” depends on the input.
- **10, Hit:** 330x at 20 bits, 610x at 24 and 710x at 28 on semiprimes, where step 5 runs once.
    - The ratio is 35x at 8 bits and 2,200x at 16 bits, where the gcd finds a factor, and it grows with n.
- **11, Untested:** memory was not measured.
    - Needs a peak-memory run, or a line saying it was not measured.
- **12, Hit:** AKS had 0 wrong in 10,106 timed results and Miller-Rabin 0 in 46,500. Fermat was wrong on Carmichael numbers (0.96 at k = 9), strong pseudoprimes and p(2p − 1), never on random composites.
    - Miller-Rabin had 3 errors in 10^6 on the hard families at k = 9.

5. What surprised you

1. **Headline: the 2^32 cliff.**
   We expected smooth growth with bit length. For AKS on balanced semiprimes, the time rose from about 15.7 ms at 32 bits to about 264 ms at 33 bits, roughly 17x, while r changed only from about 1050 to about 1070. Per coefficient operation the cost rose from about 0.45 ns to about 7 ns. Our best explanation is a quirk in implementation. Below 2^32 the product of two coefficients fits in 64 bits, so each output coefficient is computed as an accumulated dot product with a single reduction. Above 2^32 each product needs a 128-bit multiply and a 128-by-64-bit remainder. We confirmed it by instrumenting r, the number of squarings and the number of values of a: none of them jumps at 33 bits, so the extra cost is per operation. Miller-Rabin shows the same step at 33 bits but only 1.7x, because it does a few hundred multiplies per call rather than millions. A micro-benchmark explains only +12% of that jump, so part of it remains unexplained. This is an implementation artefact rather than a property of AKS, which is itself the lesson: word size can matter as much as asymptotic complexity at these input sizes.
2. **Second surprise: Fermat's wrong answer is also its slow answer.**
   We expected Fermat to be marginally faster than Miller-Rabin on every input, as it does less work per round. On Carmichael numbers it is about 7x slower (3.06 μs against about 420 ns at k = 9), because it treats them as primes and runs every round while confidently returning the wrong answer. Raising k cannot help, since the share of fooling bases is 1.0000 above 40 bits. Miller-Rabin's square-root check rejects them in round 1, so being more careful also made it faster here.
3. **Third surprise: the 1/4 bound is actually tight.**
   We expected the textbook error bound of 1/4 per round to be loose in practice. The p(2p − 1) family reaches 0.2500 at k = 1 and 3.70 x 10^-6 at k = 9 against a bound of 3.81 x 10^-6, and the worst n up to 20,000 is 12403 = 79 · 157 at 24.5%.

6. What we'd do next

A natural extension would be expansion to an arbitrary number of bits. Currently we are limited to 64 bit numbers. These give us a range from 0 to over 18 quintillion, and any trend or outlier numbers easily fit within this bound. However, real world applications such as RSA key generation need primes of 1024 bits or more, and cryptographic protocols commonly use primes between 256 and 4096 bits. In most hardware architectures, 64 bit integers are the largest number one CPU instruction can perform a full operation on. Testing on larger numbers would require support of multi-word numbers, where multiplication costs grow by O(b^2) for b-bit inputs with regular arithmetic, or O(b^1.585) with the Karatsuba algorithm. Randomized test runtime would grow roughly cubically and AKS would not be practical at all. Extrapolating our fitted AKS time of about b^6.8 (primes of 12-26 bits) to 1024 bits gives roughly 10^4 years, and over 10^6 years once multi-word arithmetic is counted. The extrapolation is far outside the fitted range and is indicative only.

We would also want to investigate ways to speed up the AKS algorithm. Currently we just implement regular polynomial squaring algorithms with O(r^2) operations. In the research for this project we came across faster polynomial multiplication methods which reduce this to O(r log r) which would lower runtime and operation count. We would also see if we could remedy the jump in runtime at 32 bits using Montgomery multiplication. If this would make AKS viable for the real-world uses listed above would have to be determined with the improved implementation.

We would replace schoolbook squaring with Karatsuba or an NTT-based polynomial multiplication (O(r log r) per square), run the independent values of a on separate threads, and apply the parameter choices of Lenstra and Pomerance to reduce the size of r. We would also replace the 128-bit remainder with Montgomery multiplication, which should remove the cliff at 2^32 in AKS and, we expect, the 33-bit step in Miller-Rabin. Finally, we would run all three algorithms from the same instance files at every size, instead of the reference exponentiation we used for some comparisons.

For Miller-Rabin specifically, we would swap % n for Montgomery multiplication and check whether the 33-bit step and the 2 times spread between primes disappear. Fermat and AKS are already run on the same instance files in the final benchmark, so the comparison with them is direct rather than made through a reference exponentiation.KS viable for real world use cases listed above would have to be determined with this improved implementation.

7. Individual contributions

Ben

I was the primary author of the AKS implementation. Beyond it, I built the shared benchmark harness and trial-division oracle, the root Makefile and repository structure, the AKS test suite, the AKS experiments and plots, part of the AI-use change log (HELPED.md), and the AKS sections of the report and video. Much of the code was drafted by an AI coding agent; My role was directing it, checking its output against the oracle and timing runs, and finding and fixing its errors.

Brad

I created the initial repo and organized how code would be combined. I also worked on the Miller-Rabin implementation and randomized testing as well as section 2 (Theoretical comparison) and section 5 (What we'd do next) alongside predictions and various other work in the other sections. I wrote all Miller-Rabin specific sections as well.

CJ

I worked on implementation of the Fermat Primality algorithm and the shared writing of section 1 (The problem and the algorithms), section 3 (Experimental Methodology), and section 6 (What we'd do next), as well as the Fermat specific section of all of the document. I combined all of our algorithm and testing branches into one unified branch to test all algorithms against each other and performed testing on my computer.

8. AI use

AI was used in the creation of the majority of code, including algorithms, testing scripts, and assistance in creating the diagrams. The majority of writing in this document and in the video were human written with some assistance with grammar, word choice, and importing figures.

AI was especially useful in combining our 3 separate algorithm branches with different testing methodologies into a single unified testing suite. Working over these 3 branches and figuring out the best approach to testing and the best way to work each together manually would have been an enormous effort requiring juggling a lot of information simultaneously. Using an LLM made this much easier as it was able to break down the problem into smaller components and allowed a back and forth of determining what design decisions would go into the final unified project.

References

Agrawal, M., Kayal, N., Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics* 160(2), 781–793.

Miller, G. L. (1976). Riemann's hypothesis and tests for primality. *Journal of Computer and System Sciences* 13(3), 300–317.

Rabin, M. O. (1980). Probabilistic algorithm for testing primality. *Journal of Number Theory* 12(1), 128–138.

Alford, W. R., Granville, A., Pomerance, C. (1994). There are infinitely many Carmichael numbers. *Annals of Mathematics* 139(3), 703–722.

Lenstra, H. W., Pomerance, C. (2019). Primality testing with Gaussian periods. *Journal of the European Mathematical Society* 21(4), 1229–1269.